

Introducción

El código original (`codigo_original.py`) es un programa sencillo que busca los números primos entre 1 y 100000. Para cada número la función `es_primo(n)` comprueba si es divisible por cualquier número desde 2 hasta `n-1`. Si encuentra un divisor, descarta el número; si no, lo considera primo. Luego se usa una comprensión de listas para ir recogiendo todos los primos y al final se muestra la cantidad y el tiempo que ha tardado.

Problemas del `codigo_original.py`

1. El bucle es demasiado largo, la función `es_primo(n)` itera prácticamente hasta `n`, lo cual es innecesario. Por ejemplo, para saber si 100.000 es primo, llega a hacer casi **100.000 divisiones**. Eso consume muchísimo tiempo cuando el rango es grande como en este caso.
2. No se aprovecha la raíz cuadrada, en matemáticas, si un número no tiene divisores hasta su raíz cuadrada, ya es primo. El código original ignora esta propiedad, con lo que hace el doble de trabajo del necesario, practicamente va número por número.
3. El código trabaja con listas y bucles tradicionales, sin aprovechar la velocidad de NumPy ni las operaciones vectorizadas que podrían acelerar.

Optimización

Para optimizar el código se usó 3 técnicas:

- Reducir el rango del bucle: Iterar solo hasta la raíz cuadrada de `n`.

En la función que verifica si un número es primo, en lugar de iterar desde 2 hasta `n-1`, cambié el bucle para que solo llegue hasta la raíz cuadrada de `n` (en concreto, hasta `int(n**0.5) + 1`). Porque si un número no es primo, tiene un divisor menor o igual a su raíz cuadrada.

- List comprehensions: Para mejorar la eficiencia en la creación de listas.

El código original ya usaba una list comprehension para generar la lista de primos: `[num for num in range(1, 100001) if es_primo(num)]`, así que se mantuvo.

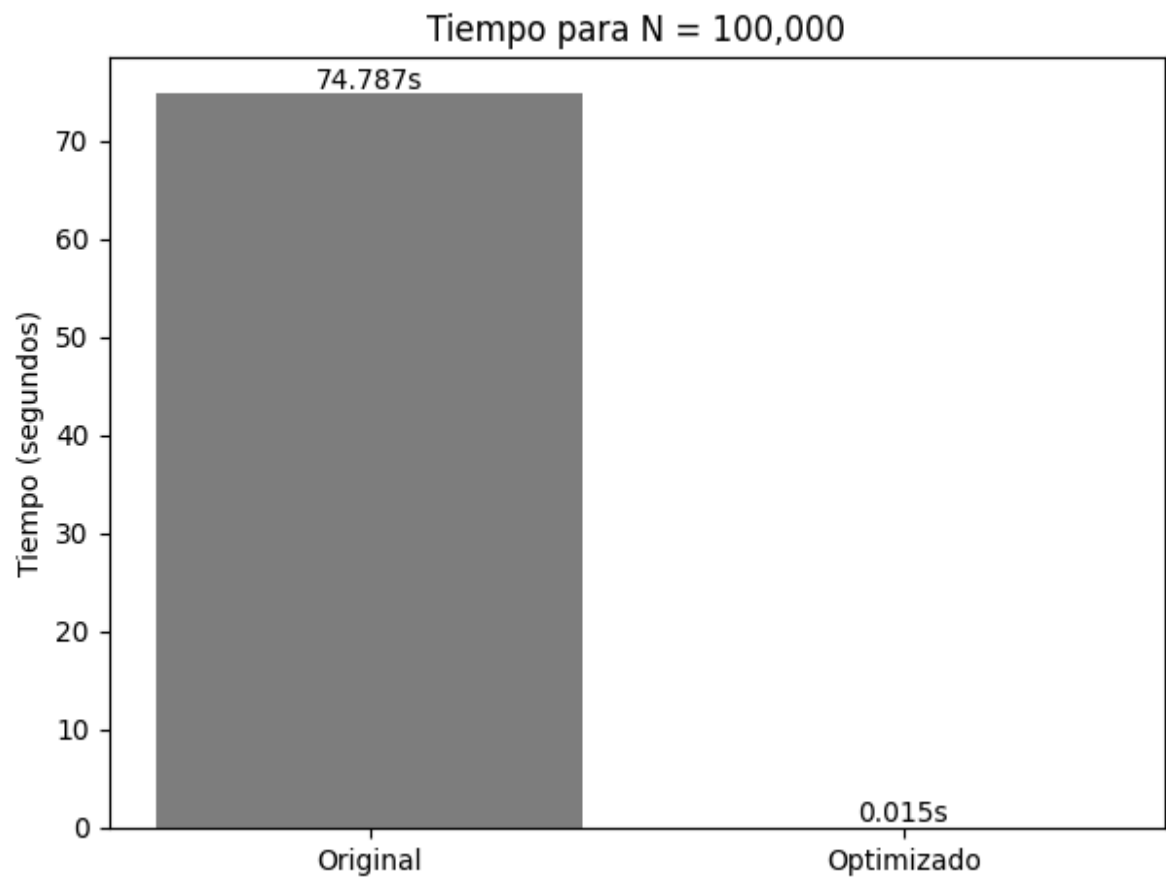
- NumPy: Utilizar arrays para acelerar las operaciones.

Reemplacé completamente la función `es_primo` por un algoritmo de criba de Eratóstenes usando arrays de NumPy. En lugar de preguntar número por número, creo un array de booleanos del tamaño del límite, marco como False los múltiplos de cada primo empezando desde 2, y al final recojo los índices que quedaron como True.

Resultados

Comparación de tiempos de ejecución

Versión	Tiempo (segundos)
Original (sin optimizar)	73.22532 s
Optimizado (raíz cuadrada + list comprehension + NumPy)	0.0166 s



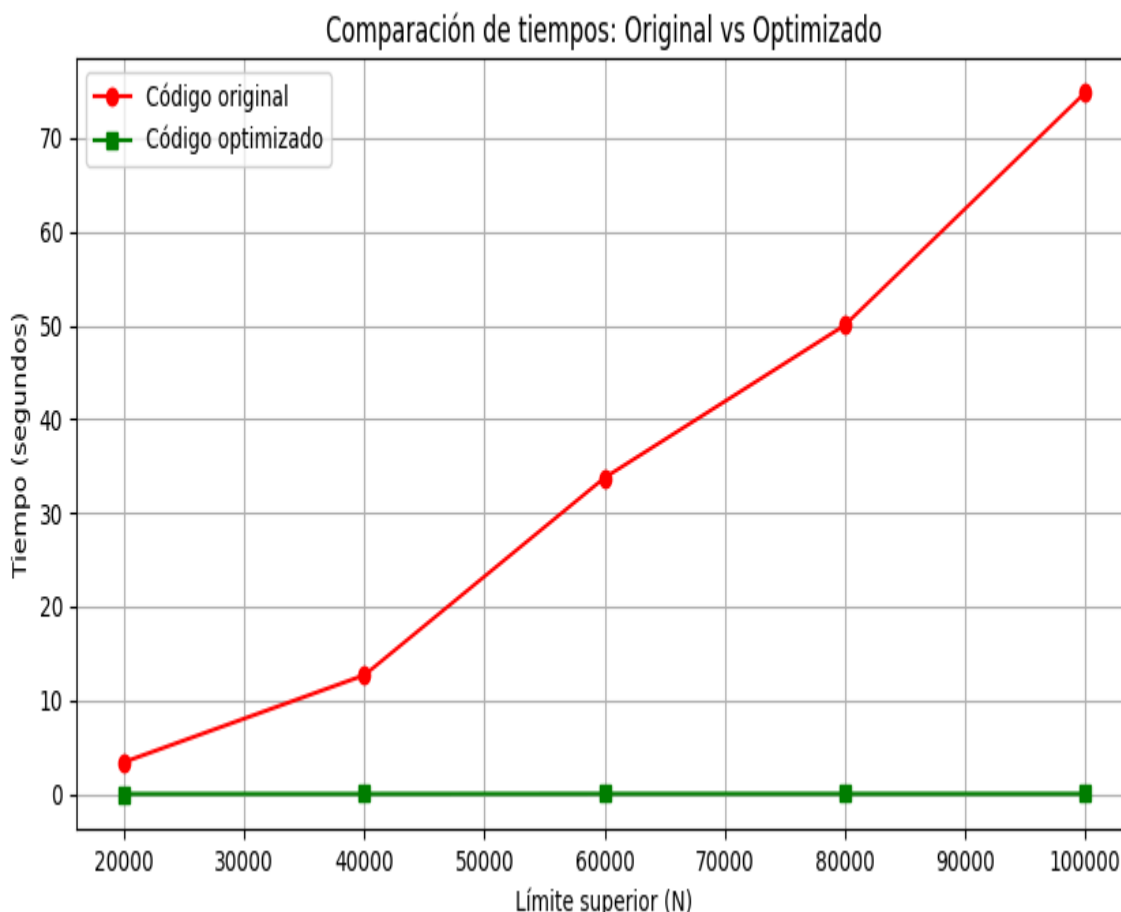
La mejora es evidente, el código es aproximadamente 4.400 veces más rápido que el original, el cual pasa de practicamente 1 minuto a tan solo 16 milésimas de segundo.

Resultados de Profiling (Optimizado)

Total: 7 llamadas de función en 0.003 segundos

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.002	0.002	0.002	0.002	profiling.py:17(primos_optimizado)
1	0.000	0.000	0.003	0.003	{built-in method builtins.exec}
1	0.000	0.000	0.000	0.000	numeric.py:171(ones)
1	0.000	0.000	0.002	0.002	{:1()}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_Isprof.Profiler' objects}
1	0.000	0.000	0.000	0.000	{built-in method numpy.empty}
1	0.000	0.000	0.000	0.000	multiarray.py:1106(copyto)

- Solo 7 llamadas a funciones en total, lo que demuestra que el código optimizado es muy directo y no incurre en gastos innecesarios.
- La función `primos_optimizado` concentra **0.002** segundos de tiempo interno (**`tottime`**).
- Las operaciones de NumPy como `ones`, `empty` y `copyto` aparecen pero con tiempos prácticamente nulos (**0.000 segundos**), porque NumPy trabaja internamente en C y las llamadas desde Python son muy rápidas.
- No hay bucles lentos en Python: el trabajo pesado lo hace `es_primo[i*i:limite+1:i] = False`, que apenas se refleja en el profiling de Python



Conclusiones

Las 3 técnicas aplicadas funcionan muy bien, la que mayor impacto tiene es la de NumPy, el resto ayuda.

Beneficios

- Rendimiento increíble ya que, se pasó de 73 segundos a 0.0166 segundos y eso es más de 4.400 veces más rápido.
- Menos consumo de CPU, al tardar menos el programa usa muchos menos recursos. Si tuviera que ejecutarse muchas veces, el ahorro sería total.
- Código más limpio con NumPy, puede resultar más compleja de entender al principio.

Recomendaciones

- Usar NumPy cuando los datos sean numéricos y grandes, Python puro se queda corto y NumPy es la opción.
- Aplicar la raíz cuadrada siempre que compruebes divisores ahorra muchísimas iteraciones innecesarias.
- El cProfile me ayudó a ver que en el código optimizado las funciones de Python apenas consumen tiempo. Es una buena práctica para no optimizar donde no hace falta.

<https://github.com/Michael07-mn/Buenas-practicas-Programacion.git>